

图论

图论是NOIP常考内容，而且一般都是以比较难的题的形式出来的。难度一般不在于图论的算法，考查的一般都是些图论的基础算法，主要难度在于如何建图，如何对给定的图进行转换。图论常考的算法有：

- (1) 并查集
- (2) 最短路
- (3) 最小生成树
- (4) 缩点

并查集

并查集在NOIP, 各省省选, NOI中考查的次数还是比较多的。很多时候只是其中的部分算法。

洛谷	3958	noip2017	奶酪
洛谷	2661	noip2015	信息传递
洛谷	1967	noip2013	货车运输
洛谷	1525	noip2010	关押罪犯
洛谷	6185	NOI onlie	序列
洛谷	4768	NOI 2018	归程
洛谷	2178	NOI 2015	品酒大会

并查集

常见的并查集的维护方法有两种：

启发式合并（按秩合并）和路径压缩

启发式合并效率比较低，最坏时间复杂度为 $n\sqrt{n}$ ，但是一般达不到，我们可以认为是常数比较大的 $n\log n$ ，但是启发式合并的优势在于每次合并只会更新其中一个集合的祖先的父亲结点，而其他结点的父亲结点不会发生改变，所以如果要撤销是很方便的，因为每次构成的集合树变化小。路径压缩效率特别高，可以近似认为是 $O(n)$ 的，但是路径压缩每一次合并这条路径上很多点的父亲结点都会发生变化，直接指向其祖先，所以每一次合并集合树的形态就会发生很大改变，几乎没有办法撤销或者还原。

注意，两者不能同时使用。

路径压缩

```
int getf(int x)
{
    if(x==father[x])//如果找到祖先
        return x;
    if(father[x]!=x)//如果不是祖先
    {
        father[x]=getf(father[x]);//更新他的父亲为祖先
        return father[x];//继续往上面找
    }
}

int t1=getf(u);
int t2=getf(v);//分别找到这两个强盗的最大boss
if(t1!=t2)
    father[t1]=t2;//把这两个boss合并了。
```

启发式合并

```
int getf(int x)
{
    if(x==fa[x])
        return x;
    return getf(fa[x]);
}
int f1=getf(x);
int f2=getf(y);
if(f1!=f2)//不在一个集合
{
    if(deep[f1]<deep[f2])
    {
        int tmp=f1;f1=f2;f2=tmp;
    }
    fa[f2]=f1;//深度小的往大的合并
    if(deep[f1]==deep[f2])
        deep[f1]++;
}
```

奶酪

现有一块大奶酪，它的高度为 h ，它的长度和宽度我们可以认为是无限大的，奶酪中间有许多半径相同的球形空洞。我们可以在这块奶酪中建立空间坐标系，在坐标系中，奶酪的下表面为 $z = 0$ ，奶酪的上表面为 $z = h$ 。

现在，奶酪的下表面有一只小老鼠 Jerry，它知道奶酪中所有空洞的球心所在的坐标。如果两个空洞相切或是相交，则 Jerry 可以从其中一个空洞跑到另一个空洞，特别地，如果一个空洞与下表面相切或是相交，Jerry 则可以从奶酪下表面跑进空洞；如果一个空洞与上表面相切或是相交，Jerry 则可以从空洞跑到奶酪上表面。

位于奶酪下表面的 Jerry 想知道，在不破坏奶酪的情况下，能否利用已有的空洞跑到奶酪的上表面去？

空间内两点 $P_1(x_1, y_1, z_1)$ 、 $P_2(x_2, y_2, z_2)$ 的距离公式如下：

$$\text{dist}(P_1, P_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$



奶酪

如果要从奶酪下底面跑到上底面，那么说明下底面和上底面只有有一个洞是连通的，所以我们可以用并查集维护连通性，把所有连通的洞放入一个集合中，最后判断是否有下底面的洞所在集合和上底面的洞所在集合相同



信息传递

有 n 个同学（编号为 1 到 n ）正在玩一个信息传递的游戏。在游戏里每人都有一个固定的信息传递对象，其中，编号为 i 的同学的信息传递对象是编号为 T_i 的同学。

游戏开始时，每人都只知道自己的生日。之后每一轮中，所有人会同时将自己当前所知的生日信息告诉各自的信息传递对象（注意：可能有人可以从若干人那里获取信息，但是每人只会把信息告诉一个人，即自己的信息传递对象）。当有人从别人口中得知自己的生日时，游戏结束。请问该游戏一共可以进行几轮？

信息传递

该题本质就是最有向图的最小环，我们可以建图，也可以直接跑并查集，如果信息由A指向B，那么就连一条A到B的边，让A的父亲指向B，同时路径长度+1。

在合并过程中，如果两个结点的祖先节点相同，那么就说明构成了一个环，长度为两个点到祖先节点的长度之和+1。

关押罪犯

S 城现有两座监狱，一共关押着 N 名罪犯，编号分别为 $1 \sim N$ 。他们之间的关系自然也极不和谐。很多罪犯之间甚至积怨已久，如果客观条件具备则随时可能爆发冲突。我们用“怨气值”（一个正整数值）来表示某两名罪犯之间的仇恨程度，怨气值越大，则这两名罪犯之间的积怨越多。如果两名怨气值为 c 的罪犯被关押在同一监狱，他们俩之间会发生摩擦，并造成影响力为 c 的冲突事件。

每年年末，警察局会将本年内监狱中的所有冲突事件按影响力从大到小排成一个列表，然后上报到 S 城 Z 市长那里。公务繁忙的 Z 市长只会去看列表中的第一个事件的影响力，如果影响很坏，他就会考虑撤换警察局长。

在详细考察了 N 名罪犯间的矛盾关系后，警察局长觉得压力巨大。他准备将罪犯们在两座监狱内重新分配，以求产生的冲突事件影响力都较小，从而保住自己的乌纱帽。假设只要处于同一监狱内的某两个罪犯间有仇恨，那么他们一定会在每年的某个时候发生摩擦。

那么，应如何分配罪犯，才能使 Z 市长看到的那个冲突事件的影响力最小？这个最小值是多少？

关押罪犯

这是一道权值并查集的裸题，该题合并的时候是有点值的。

首先，因为要保证最大的怒气最小，所以我们可以贪心，尽量把怒气值最大的两个人隔开。

那么我们如何用并查集维护呢？因为两个人不在一个监狱中，所以我们可以分别用两个并查集来维护两个集合，如果在某次合并的过程中发现这两个人已经在同一个集合中了，那么说明矛盾了，就输出当前答案。

因为是已经排好序的，所以此时的答案一定是最优解。



星球大战

很久以前，在一个遥远的星系，一个黑暗的帝国靠着它的超级武器统治着整个星系。

某一天，凭着一个偶然的机遇，一支反抗军摧毁了帝国的超级武器，并攻下了星系中几乎所有的星球。这些星球通过特殊的以太隧道互相直接或间接地连接。

但好景不长，很快帝国又重新造出了他的超级武器。凭借这超级武器的力量，帝国开始有计划地摧毁反抗军占领的星球。由于星球的不断被摧毁，两个星球之间的通讯通道也开始不可靠起来。

现在，反抗军首领交给你一个任务：给出原来两个星球之间的以太隧道连通情况以及帝国打击的星球顺序，以尽量快的速度求出每一次打击之后反抗军占据的星球的连通块的个数。（如果两个星球可以通过现存的以太通道直接或间接地连通，则这两个星球在同一个连通块中）。





星球大战

并查集是只能做合并不是做删除的，所以正着我们是没有办法用并查集做的，所以考虑反过来做，我们就可以把摧毁看做重建，先把摧毁之后最后的答案算出来，然后再离线反向算出倒着每一次重建之后的连通状态。



图论

Tom 热爱出毒瘤题，但这次他似乎出了一个并不毒瘤的题。

给定一张 N 个点 M 条无向边的图，每条边有边权 w 。定义一个连通块的大小为该连通块内所有边权的和。

然后进行 K 组询问。每次询问在由所有边权不超过 x_i 的边构成的新的生成子图中连通块的大小的最大值。

【输入格式】

从文件 `Graph.in` 中读入数据。

第 1 行 3 个整数 n, m, k 表示图有 n 个点， m 条边。有 k 组询问。

第 2 行到第 $m+1$ 行每行三个正整数 u, v, w ，表示点 u 和点 v 之间有一条边权为 w 的边。

第 $m+1$ 行到第 $m+k$ 行每行一个整数 x ，表示在这次询问中，只能使用边权不超过 w 的边。

可能存在重边，但不存在自环。

【输出格式】

输出到文件 `Graph.out` 中。

K 行，对于每组询问输出一行答案。

对于全部测试数据，满足 $N, M, K \leq 100000$ ， $w \leq 1000$

图论

暴力做，对于每一次询问，做一遍并查集，时间复杂度为 $k*m$ 。

其实只要认真看一下数据范围就可以知道怎么优化，因为 w 很小，所以很多询问都是重复的，所以我们可以直接枚举预处理出来 $[1, 1000]$ 中所有边权的答案，这样时间复杂度就和 k 没有关系了。

我们需要考虑 w 和 $w-1$ 的关系，按 w 构图肯定比按 $w-1$ 构图需要的边更多，所以我们先把线段从小到大排序，然后依次加入边，如果 w 和 $w-1$ 之间没有新边加入，那么答案直接继承，如果多了一条边那么直接加上该边的贡献。

游戏

题目描述

lxhgww最近迷上了一款游戏，在游戏里，他拥有很多的装备，每种装备都有2个属性，这些属性的值用 $[1, 10000]$ 之间的数表示。当他使用某种装备时，他只能使用该装备的某一个属性。并且每种装备最多只能使用一次。游戏进行到最后，lxhgww遇到了终极boss，这个终极boss很奇怪，攻击他的装备所使用的属性值必须从1开始连续递增地攻击，才能对boss产生伤害。也就是说一开始的时候，lxhgww只能使用某个属性值为1的装备攻击boss，然后只能使用某个属性值为2的装备攻击boss，然后只能使用某个属性值为3的装备攻击boss……以此类推。现在lxhgww想知道他最多能连续攻击boss多少次？

游戏

该题可以用二分图来做，也可以直接用并查集，我们把一个武器的两个属性分别看做点。对于新来的武器的两个属性，我们先判断一下是否他们已经在在一个集合内了，如果在，那么我们把大的那个点标记，说明这两个属性都可以到达；如果不在一个集合，说明这两个属性暂时只能使用一个，我们肯定贪心的选择属性小的那一个，所以我们这次只标记属性小的一个，并让属性小的父亲指向属性大的一个，那么下一次标记的时候标记祖先就可以了。最后枚举一遍看下能从1连续到哪里。

最短路

常见的求图论最短路的办法有三种：

(1) Floyd求任意点对的最短路。时间复杂度 n^3 ，虽然还是比 n 遍Dijkstra或者SPFA慢，但是主要在于代码短。

(2) Dijkstra求单源最短路，使用堆优化，时间复杂度稳定在 $n\log n$ ，可以求出一个点到其他所有点的最短路，但是边权不能有负数，更不能有负环。

(3) SPFA求单源最短路，使用双端队列优化，时间复杂度不是很稳定，一般在 $n\log n$ ，可以被卡到 n^2 ，主要是可以处理负边权，判断是否有负环。

Floyd

```
for(int k=1;k<=n;k++)//枚举中间节点
{
    for(int i=1;i<=n;i++)//枚举起点
    {
        for(int j=1;j<=n;j++)//枚举终点
        {
            dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
        }
    }
}
```

Dijkstra

```
void dijkstra(int s)
{
    for(int i=1;i<=n;i++)
        dis[i]=inf; // 初始化最小值为inf
    dis[s]=0; // 起点到起点最短路为0
    q.push(point(s,0));
    while(!q.empty())
    {
        int u=q.top().id;
        q.pop();
        if(book[u]) continue;
        book[u]=1; // 出队标记
        for(int i=head[u];i>0;i=edge[i].next)
        {
            int v=edge[i].to;
            if(dis[v]>dis[u]+edge[i].w&&!book[v])
            {
                dis[v]=dis[u]+edge[i].w;
                q.push(point(v,dis[v]));
            }
        }
    }
}
```

SPFA

```
for(int i=0;i<=n;i++) dis[i]=inf;
int h=maxn,t=maxn;
q[h]=s;t++;dis[s]=0;vis[s]=1;
while(h<t){
    int u=q[h];h++;
    cnt[u]++;
    if(cnt[u]>=m){//存在负环
        puts("No");
        exit(0);
    }
    vis[u]=0;
    for(int i=head[u];i;i=e[i].next){
        int v=e[i].to;
        if(dis[v]>dis[u]+e[i].w){
            dis[v]=dis[u]+e[i].w;
            if(!vis[v]){
                if(dis[v]<dis[q[h]]){
                    h--;//放队首
                    q[h]=v;
                }
                else{//放队尾
                    q[t]=v;
                    t++;
                }
            }
            vis[v]=1;
        }
    }
}
```

灾后重建

题目背景

展开

B 地区在地震过后，所有村庄都造成了一定的损毁，而这场地震却没对公路造成什么影响。但是在村庄重建好之前，所有与未重建完成的村庄的公路均无法通车。换句话说，只有连接着两个重建完成的村庄的公路才能通车，只能到达重建完成的村庄。

题目描述

给出 B 地区的村庄数 N ，村庄编号从0到 $N - 1$ ，和所有 M 条公路的长度，公路是双向的。并给出第 i 个村庄重建完成的时间 t_i ，你可以认为是同时开始重建并在第 t_i 天重建完成，并且在当天即可通车。若 t_i 为0则说明地震未对此地区造成损坏，一开始就可以通车。之后有 Q 个询问 (x, y, t) ，对于每个询问你要回答在第 t 天，从村庄 x 到村庄 y 的最短路径长度为多少。如果无法找到从 x 村庄到 y 村庄的路径，经过若干个已重建完成的村庄，或者村庄 x 或村庄 y 在第 t 天仍未重建完成，则需要返回-1。

对于100%的数据，有 $N \leq 200$ ， $M \leq N \times (N - 1)/2$ ， $Q \leq 50000$ ，所有输入数据涉及整数均不超过100000。

灾后重建

如果对于每一个询问跑一遍最短路，询问特别多，并且边数也不少，所以会超时。

该题是一道很好的加深对Floyd理解的题，主要在于理解Floyd最外层循环的含义。最外层枚举的 k 表示经过的路径只有前面 k 个点中的某些点

我们可以对时间 t 进行排序，然后根据时间来确定中转点，如果一个村庄修好，那么这个村庄就可以作为中转点，来判断是否可以更新起点到终点的路径，因为最开始路径都是 inf ，所以中间存在 inf 的值说明还没有连通。

做的时候只需要把Floyd最外层的循环变为按时间 t 排好序的点的循环就可以了。

收费站

题目描述

在某个遥远的国家里，有 n 个城市。编号为 $1, 2, 3, \dots, n$ 。这个国家的政府修建了 m 条双向的公路。每条公路连接着两个城市。沿着某条公路，开车从一个城市到另一个城市，需要花费一定的汽油。开车每经过一个城市，都会被收取一定的费用（包括起点和终点城市）。所有的收费站都在城市中，在城市间的公路上没有任何的收费站。小徐现在要开车从城市 U 到城市 v ($1 \leq u, v \leq n$)。她的车最多可以装下 s 升的汽油。在出发的时候，车的油箱是满的，并且她在路上不想加油。在路上，每经过一个城市，她要交一定的费用。如果她某次交的费用比较多，她的心情就会变得很糟。所以她想知道，在她能到达目的地的前提下，她交的费用中最多的一次最少是多少。这个问题对于她来说太难了，于是她找到了聪明的你，你能帮她吗？

对于 100% 的数据，满足 $n \leq 10000$ ， $m \leq 50000$ ， $s \leq 10000000000$

对于 100% 的数据，满足 $ci \leq 10000000000$ ， $fi \leq 10000000000$ ，可能有多条边连接着相同的城市。

收费站

题目中有明显的最大值最小，所以可以考虑二分答案。二分他经过收费站时教过的最多的一次的费用，然后跑最短路，跑的时候只跑边权小于假设值的边，看下是否可以到达终点。如果可以到达终点，再判断油够不够，如果跑最短路油都不够那么也是不合理的。

圣诞树

题目描述

圣诞节到了，FireDancer 准备做一棵大圣诞树。右图为圣诞树的一个简单结构。这棵树被表示成一组被编号的结点和一些边的集合。结点从 1 到 n 编号。树的根永远是 1。每个结点都有一个自身特有的数值，称为它的重。各个结点的重可能不同。对于一棵做完的树来说，每条边都有一个价值，若设这条边 e 连接结点 i 和结点 j ，且 i 为 j 的父结点(根是最老的祖先)，则该边的价值为 $(j$ 的所有子孙及它自己的重之和) * (e 的单位价值 ce)。现在 FireDancer 想造一棵树，使得树上所有边的总价值最小，并且所有的点都在树上，因为 FireDancer 喜欢大树。

第一行两个整数 n 和 m ($0 \leq n, m \leq 50000$)，表示结点总数和可供选择的边数。

下面一行有 n 个整数，依次表示每个结点的重。

下面 m 行，每行有 3 个正整数 a, b, c ，表示结点 a 和结点 b 之间有一个单位价值为 c 的边可供你造树时选择。

输入中的所有数都小于 2^{16} 。

圣诞树

题目中最开始给的是一幅图，让你从中选部分边构成一棵树，假设我们把所有点都选好了，计算的时候，如果按照给定的公式按边去计算就很麻烦，我们可以把公式转换一下，把边的贡献转换成点的贡献。实际上我们就会发现所有点的贡献就是该点的重*该点到根结点的路径长度。无论怎么选边，最终点都是固定的，所以我们只需要保证任意一个结点到根结点的路径最短就可以了，所以该题实际上就是一道最短路的变形题。

最快路线

题目描述

精明的小R每每开车出行总是喜欢走最快路线,而不是最短路线.很明显,每条道路的限速是小R需要考虑的关键问题.不过有一些限速标志丢失了,于是小R将不知道能开多快.不过有一个合理的方法是进入这段道路时不改变速度行驶.你的任务就是计算从小R家(0号路口)到D号路口的最快路线.

现在你得到了这个城市的地图,这个地图上的路都是单向的,而且对于两个路口A和B,最多只有一条道路从A到B.并且假设可以瞬间完成路口的转弯和加速.

第一行是三个整数N,M,D(路口数目,道路数目,和目的地). 路口由0...N-1标号

接下来M行,每行描述一条道路:有四个整数A,B,V,L,(起始路口,到达路口,限速,长度) 如果V=0说明这段路的限速标志丢失.

开始时你位于0号路口,速度为70.

对于100%的数据, $2 \leq N \leq 150$; $M \leq N^2$, $0 \leq V \leq 500$; $1 \leq L \leq 500$

最快路线

因为道路的限速不一样，所以不能直接跑最短路，因为距离的长短不代表时间的长短，但是该题数据中 n 比较小，速度也比较小，所以我们可以看做是二维的最短路，以前都是直接记录 $dis[i]$ 表示 i 点到起点的最短路，这里用 $dis[i][j]$ 表示 i 点在 j 速度下到起点的最短时间。因为一个点可以由不同的点到达，所以我们还需要记录一下在当前最短路的情况下该点的上一个点的速度和编号。更新的时候判断同样在该速度的时间下的最短路。

Elaxia 的路线

题目描述

展开

最近，Elaxia 和 w^{**} 的关系特别好，他们很想整天在一起，但是大学的学习太紧张了，他们必须合理地安排两个人在一起的时间。

Elaxia 和 w^{**} 每天都要奔波于宿舍和实验室之间，他们 希望在节约时间的前提下，一起走的时间尽可能的长。

现在已知的是 Elaxia 和 w^{**} 所在的宿舍和实验室的编号以及学校的地图：

地图上有 n 个路口， m 条路，经过每条路都需要一定的时间。具体地说，就是要求无向图中，两对点间最短路的最长公共路径。

Elaxia 的路线

首先，我们要想到这条最长公共路径一定是连续的，这个很好证明，中间不连续的部分，两个人的路径一定有长有短，那么肯定都走长的那个肯定最优。

所以很容易想到一种暴力的做法就是求出所有点对的最短路，但是最短路路径不止一条，所以我们记录所有到起点和终点路径长度都等于当前最短路的点，并且标记，这样所有可能的情况都标记出来了，这些点一定在两个人的最短路径上，然后我们再枚举这些点，找一个两点之间距离最大的，但是时间复杂度是 n^3 或者 $n^2 \log n$ ，过不了。

Elaxia 的路线

我们需要结合最短路的性质来看，跑完最短路之后，从起点看，这幅图就是一个DAG(有向无环图)，所以我们没有必要计算任意两点之间的最短路径，只需要知道起点到任意点的最短路就可以了，然后标记一下所有可能在最短路路上出现的点，时间复杂度为 n^2 。

因为是有向无环图，所以我们只需要保留只出现在两幅DAG中的有向边就可以了。最后跑一遍TP排序就可以得到答案了。



寻找道路

在有向图 G 中，每条边的长度均为 1，现给定起点和终点，请你在图中找一条从起点到终点的路径，该路径满足以下条件：

1. 路径上的所有点的出边所指向的点都直接或间接与终点连通。
2. 在满足条件1的情况下使路径最短。

注意：图 G 中可能存在重边和自环，题目保证终点没有出边。

请你输出符合条件的路径的长度。



寻找道路

该题不仅仅是找最短路，而是在招找最短路的基础上还要满足所有点出边指向的点都直接或间接和终点相连，很明显直接找最短路是错误的。

既然要求所有的点出边指向的点都和终点相连，那么我们先反着建一遍图，记录从终点出发能到达的所有点。接着再正着跑最短路，判断某个点能到达的点是否都被标记过。，然后跑一遍最短路就可以了。

最小生成树

常见的最小生成树算法有两个：Kruskal和Prim算法

(1) Kruskal的思路是不停的往生成树中加边，对于一幅图中，最小生成中最短的边肯定是要选的，第二短的边也要选，第三短的边选或者不选就看是否构成了环，所以kruskal就是贪心+并查集，效率 $m \log m$ (排序)

(2) Prim的思路是不停的往生成树中加点，对于当前已经构造好的生成树，剩下的点中离生成树最近的点的这条边是一定要选的，因为没有其他边可以优化他，思路和dijkstra跑最短路的思路一样，效率 $n \log n$ 。

最大生成树

Kruskal可以直接求最大生成树，把边按从大到小排序就可以，Prim如果要求最大生成树，如果维护一个大根堆是错误的，他只能把每条边取反，变为负数，然后仍然求最小生成树，最后再把答案取反。

保留道路

题目描述

很久很久以前有一个国家，这个国家有N个城市，城市由1,2,3,...,N标号，城市间有M条双向道路，每条道路都有两个属性g和s，两个城市间可能有多条道路，并且可能存在将某一城市与其自身连接起来的道路。后来由于战争的原因，国王不得不下令减小花费从而关闭一些道路，但是必须要保证任意两个城市相互可达。

道路花费的计算公式为 $wG \cdot \max\{\text{所有剩下道路的属性}g\} + wS \cdot \max\{\text{所有剩下道路的属性}s\}$ ，其中wG和wS是给定的值。国王想要在满足连通性的前提下使这个花费最小，现在需要你计算出这个花费。

保留道路

因为该题每条道路有两个属性，所以无论根据那个属性跑最小生成树都有可能是错误的。

我们可以先按照 g 从小到大排序，对于当前已经选择的边，我们再按照 s 从小到大排序，可以用插入排序，每次时间复杂度为 $O(n)$ 。

每新加进来一条边，我们先和以前选择的边一起按照 s 从小到大排好序，接下来再判断这 n 条边是否构成了环，如果不构成环，那么就把这条边加进来，如果构成环，先不管他，放在里面先不选，最后当我们选择了 n 个点之后，更新当前答案。

实际上就是一种选择和替换的方法，先贪心选 $n-1$ 条边，作为当前答案，接着继续选，看下是否可以替换，因为选的边按照 s 排序，所以只算前面 $n-1$ 条就可以了。

水资源

【问题描述】

D 国是一个干旱的国家，极度缺水。该国包含 n 个城市，第 i 个城市可以花费 $w[i]$ 的代价在城市内部打井从而获得水源。两个城市之间也可以耗费一定代价修建水渠来共享水源。求使所有城市都有水的最小代价

【输入格式】

第1行为一个整数： n

第2行长度为 n 的01字符串， $ch[i]='1'$ 说明 i 号城市本来有水，反之无水

第3行为 n 个整数，即 $w[i]$ ，若 $w[i]=-1$ 则表示该城市无法打井

第4行为一个整数： m

接下来 m 行，每行 a, b, v ($a \neq b, v > 0$) 三个数，表示城市 a, b 之间可消耗代价 v 来修一条水渠。

保留道路

每一个点有两种情况，第一种是自己打井，第二种情况是修水渠，所以我们要想办法把他们变成一种情况，我们新建一个超级原点，如果一个点有水，那么超级原点到该点建一条边权为0的边(不影响答案)，如果该点没水但是可以打井，我们可以把打井的花费看做修路的花费，我们同样连一条边到超级原点，边权为打井的代价，这样我们就把所有权值都统一为边权了，就可以直接跑最小生成树了。

次小生成树

我们学会了如何求解最小生成树，但是如果要求次小生成树需要如何求呢？我们可以用选择+替换的操作，我们先求出最小生成树已经最小生成树的边，然后依次枚举不在最小生成树上的边，当我们把这条边放到最小生成树上后，肯定会形成一个环，而环上肯定有一条边是需要删除的，根据贪心的思想，我们删除这个环上除了该边之外最大的一条边，那么如何去找最大的边呢？直接找很慢，我们可以考虑倍增维护，对于当前枚举的边 (u, v) ，找到树上 u, v 的最短路径以及最大的边，替换掉就可以了。

练习题

先写并查集、最短路、生成树的模板，再做前面讲过的题

MSOJ	1589	分组
MSOJ	1575	树
洛谷	1144	最短路计数
MSOJ	1513	逛公园
洛谷	2505	道路
洛谷	2573	滑雪
洛谷	2330	繁忙的都市
洛谷	2266	爱的距离